

Ecosistema de Inventario Seguro (EGI)

Grupo: Bazan Sofia, Milagros Carillo, Mendez Julian y Fernando Castro



Arquitectura General del Sistema

El ecosistema está compuesto por los siguientes servicios:



Frontend (nginx)

HTML, CSS, JS



Backend (Flask)

API containerizada dentro de Kubernetes (Minikube), corriendo en una VM Ubuntu (192.168.1.30). Maneja toda la lógica de negocio.



SQL Server (192.168.1.20)

VM Windows Server 2022. Almacena ubicación de equipos: aula, número de banco, fecha de mantenimiento y responsable.



MongoDB (192.168.1.30)

Containerizado en Minikube. Almacena el hardware de cada equipo: CPU, RAM, disco, SO, periféricos.



Active Directory / LDAP (192.168.1.10)

VM Windows Server 2022. Centraliza la autenticación de todos los usuarios del ecosistema.



pfSense (192.168.1.254)

Firewall perimetral. Único punto de entrada y salida de la red interna.



Flujo principal: nginx → pfSense → VM Minikube → Pod Flask → AD (auth) → SQL Server (ubicación) → MongoDB (hardware)

Topología de Red: Red Interna Aislada

Se evaluaron dos enfoques de red:

Criterio	Modo Bridge	Red Interna Aislada (elegida)
Tráfico obligado por firewall	No garantizado	Sí
Dependencia de la red de la facultad	Sí	No
Portabilidad para la defensa	Baja	Total

¿Por qué la Red Interna Aislada?

Se eligió la **Red Interna Aislada** porque el aislamiento es topológico: ninguna VM tiene un adaptador NAT propio. Todo el tráfico externo está obligado a pasar por pfSense.

Esto cumple de forma más fiel el requisito de perímetro obligatorio del enunciado y garantiza que el entorno funcione igual el día de la defensa sin depender de la red del laboratorio.

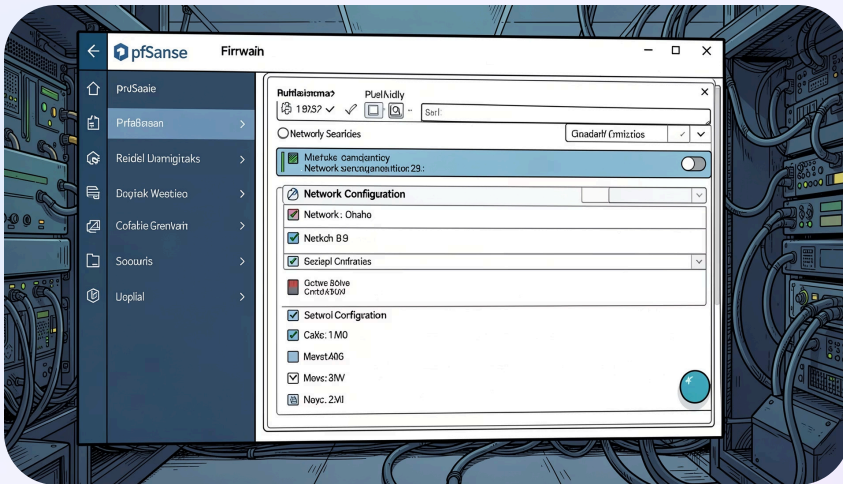
Plan de Direccionamiento IP

Todo el ecosistema opera sobre el segmento **192.168.1.0/24**. Se utilizó **direccionamiento estático manual** en todas las VMs (sin DHCP) para maximizar la previsibilidad.

Equipo / Rol	IP Estática	SO
pfSense (Gateway / Firewall)	192.168.1.254	pfSense 2.8.1
Active Directory / DNS	192.168.1.10	Windows Server 2022
SQL Server	192.168.1.20	Windows Server 2022
Minikube / Backend Flask	192.168.1.30	Ubuntu

Firewall pfSense: Configuración y NAT

pfSense 2.8.1 actúa como gateway, servidor de NAT y punto de control perimetral.



Interfaces configuradas

- **WAN (em0):** 192.168.100.136/24 – modo Puente, salida al exterior
- **LAN/SERVER (em1):** 192.168.1.254/24 – segmento interno con todas las VMs

NAT saliente

Modo Automatic Outbound NAT. Todo el segmento 192.168.1.0/24 sale enmascarado bajo la IP de la WAN. Validado con TTL 118 en pings a internet (el decremento confirma el paso por el gateway).

Base de Datos SQL Server: Modelo y Estructura

SQL Server corre en la VM Windows (192.168.1.20), instancia con nombre **ITULAB**, instalado como Express 2022 con autenticación Windows integrada al dominio itu.local.

La base de datos almacena la ubicación de cada equipo

- **equipos** – ID del equipo, tipo (desktop/laptop), número de inventario
- **ubicaciones** – aula/laboratorio, número de banco/mesa, edificio
- **responsables** – docentes o técnicos asignados al equipo
- **mantenimientos** – fechas de último y próximo mantenimiento, observaciones
- **asignaciones temporales** – alumnos o docentes con equipos asignados temporalmente

Integración con Active Directory

Los accesos a SQL Server se gestionan mediante grupos de AD, no usuarios locales. Se definió la siguiente nomenclatura de grupos:

- **Grupo_BD_Admin** → rol sysadmin en el motor
- **Grupo_BD_Inventario_C** → roles db_datareader + db_datawriter
- **Grupo_BD_Inventario_R** → rol db_datareader

Esto permite administrar accesos simplemente agregando o quitando usuarios de grupos en el AD, sin tocar SQL Server.

Base de Datos MongoDB: Modelo y Estructura

MongoDB corre containerizado dentro de Minikube en la VM Ubuntu. Se usa la imagen oficial de mongo sin Dockerfile propio.

La colección `hardware` almacena los componentes internos de cada equipo

```
{
  "equipo_id": "EQ-001",
  "fabricante": "Dell",
  "modelo": "OptiPlex 7090",
  "tipo": "desktop",
  "cpu": {
    "marca": "Intel",
    "modelo": "Core i5-11500",
    "nucleos": 6
  },
  "ram": {
    "capacidad_gb": 16,
    "tipo": "DDR4"
  },
  "almacenamiento": {
    "tipo": "SSD",
    "capacidad_gb": 512
  },
  "sistema_operativo": "Windows 10 Pro",
  "perifericos": {
    "monitor": "Dell 24\"",
    "mouse": true,
    "teclado": true
  }
}
```

Integración con SQL Server

Cada documento usa el mismo `equipo_id` que el registro en SQL Server, lo que permite al backend Flask hacer el **JOIN lógico** entre ambas bases: primero busca la ubicación en SQL Server, luego usa el ID para traer el hardware de MongoDB.

Operaciones soportadas

- Búsquedas filtradas por aula o tipo de equipo
- Actualización de componentes
- Eliminación de registros

Integración con Active Directory / LDAP

El servidor Active Directory (itu.local) en 192.168.1.10 centraliza la autenticación de todos los usuarios: profesores, técnicos y administradores.

Estructura de OUs en el AD

- **ITU/Users** → usuarios organizados por rol (Almacenes, Calidad, RRHH, etc. – adaptable a Laboratorios, Docentes, Técnicos)
- **AdminGroups** → grupos administrativos del motor de BD
- **AdminUsers** → usuarios con privilegios de administración

✓ **Ventaja clave:** un solo punto de gestión de usuarios. Agregar un nuevo técnico al sistema es tan simple como crear el usuario en el AD y agregarlo al grupo correspondiente. No hay que tocar ni Flask, ni SQL Server, ni MongoDB.

Flujo de autenticación

01

Credenciales en el frontend

El usuario ingresa sus credenciales en el frontend.

02

Consulta LDAP

El backend Flask realiza una consulta LDAP al AD en 192.168.1.10.

03

Validación

Si el usuario existe y la contraseña es válida, se le permite acceder al inventario.

Kubernetes: Despliegue Containerizado y Network Policies

El clúster Minikube se inicia con CNI Calico para habilitar Network Policies: `minikube start --cni=calico`

Estructura de manifiestos (/kubernetes)

- **namespace.yaml** – namespace aislado para el proyecto
- **deployments/mongo.yaml** – deployment de MongoDB
- **deployments/flask.yaml** – deployment del backend Flask
- **services/mongo** – servicio interno para MongoDB
- **services/sql-externo** – ExternalName apuntando a 192.168.1.20:1433
- **services/ldap-externo** – ExternalName apuntando a 192.168.1.10:389
- **network-policies/** – políticas Zero-Trust

Reglas de Network Policy implementadas (Zero-Trust)

Frontend → Flask

El Frontend puede comunicarse con Flask vía pfSense.

Flask → Bases de Datos

Flask puede comunicarse con MongoDB, SQL Server externo y LDAP externo.

MongoDB restringido

MongoDB solo acepta conexiones provenientes del Pod Flask.

Bloqueo por defecto

Todo otro tráfico dentro del namespace está bloqueado por defecto.



Esto garantiza que aunque alguien comprometa el frontend, no pueda acceder directamente a las bases de datos.